

My Model Failed Its Own Test. That Was the Point.

What I learned building a machine learning pipeline for buried pipeline inspection.

I am a physicist. I understand magnetism well. What I did not know was how to turn that knowledge into software that other people could rely on.

So I built a full machine learning pipeline to find out. It takes magnetometer readings from above a buried pipeline and produces a ranked list of places to dig.

Two things to say before I start.

The data is synthetic. Real inspection data of this type is commercial and I do not have any, so I wrote a physics-based generator instead. Every number below is measured on that generated data.

And the main model does not work. It fails its own quality check, the check runs in CI, and the job shows red. I now think this is the most useful part of the project.

The problem

Buried pipelines corrode and crack. The normal way to inspect them is to send a robot inside. But many pipelines cannot take one, because of tight bends or missing entry points.

One alternative is to walk a magnetometer along the ground above the pipe. Steel under stress changes the magnetic field around it slightly. If you can measure that change, you can guess where the damage is, without digging.

The difficulty is the size of the signal.

The Earth's magnetic field where I placed my survey is about 45,000 nT. The signal from a defect, after I remove the background, is about 25 nT. That is around 0.05 percent of the raw measurement.

So the real work is not detection. It is background removal. Everything else in the pipeline follows from that.

There is a second problem. A magnetometer cannot tell useful steel from useless steel. A buried fence post, a piece of scrap metal, or an old well casing all produce a signal that looks like a defect. If your detector only looks at signal strength, every piece of buried junk becomes a recommendation to dig. Each dig costs tens of thousands of euros.

So the task is not "find strong signals". It is "find strong signals that are actually defects".

Making the data

I could not get real data, so I wrote a forward model.

A small damaged region in a steel pipe behaves, from a distance, roughly like a magnetic dipole. The field of a dipole is standard physics and falls off as $1/r^3$. I place defects along a line, give each one a strength and a random orientation, and compute what a sensor walking above them would measure.

I should be honest about the limits. The scaling constant in my code is not the real physical constant. I chose it so the numbers land in a realistic range. I do not model the pipe body, or induced versus permanent magnetisation. The generator gives data with the right *structure*, not data a physics reviewer would accept. That was a deliberate trade: I wanted ground truth I know exactly, so I can check whether the pipeline recovers it.

The corpus is five separate 2 km lines, measured every 0.5 m, surveyed three times each. That is 15 surveys and 60,000 rows. Each line has 12 defects and 4 pieces of interference. Defects grow 15 percent between surveys, so I can also test growth prediction.

The interference is the interesting part

I wanted the false alarms to be genuinely hard, not decoration.

Interference sources sit 3 to 8 m to the side of the line, not under it. Being further away, their signal is much weaker, because of the $1/r^3$ falloff. If I left it there, they would sit below the noise and never fool anything.

So I made them stronger, by a factor of about 50. That number is not arbitrary: $(5.5/1.5)^3$ is roughly 50, which is exactly the penalty for sitting 5.5 m away instead of 1.5 m below.

The result is interference that arrives at the sensor with the *same strength* as a defect, but a different *shape*, because it comes from a different geometry.

I measured this. Defects under the pipe produce a peak about 2.2 m wide. Interference beside the pipe produces a peak about 6.2 m wide. Same height, different width.

This is the core idea of the project: **shape separates them, strength does not**. A threshold on signal strength cannot work here, even in principle.

Later analysis confirmed it in numbers. Features based on signal strength separate defects from empty ground almost perfectly. For separating defects from interference, the same features are close to useless. The features that work are shape features.

Twice my simulator fooled me

This is the part I would most want to tell someone starting out.

First mistake: my background was a polynomial, so my polynomial method looked excellent.

I built the background drift as a slow ramp plus one slow wave. I then removed it by fitting a polynomial. Of course it worked. The leftover noise came down to 5.03 nT, and my sensor noise was set to 5.0 nT. It matched exactly.

My method was not good. I was fitting a polynomial to a polynomial.

To check properly, I needed a real background. The USGS publishes magnetic observatory recordings for free. I took two days from Boulder, Colorado: one quiet day, and 10 May 2024, the largest magnetic storm in twenty years. My generator can use these real recordings instead of my invented background.

The test was useful. A simple polynomial fit leaves 6.9 to 11.3 nT of error on the storm data, well above the noise floor. It does not fit real magnetic behaviour the way it fit mine.

But my actual two-step method (a robust polynomial, then a 40 m rolling median filter) held up: about 7.8 nT on my synthetic background, 8.1 nT on the storm, 7.8 nT on the quiet day. Detection quality stayed roughly the same in all three cases.

So the pipeline survived contact with real physics. But I only know that because I went looking for a way to prove myself wrong.

Second mistake: my two-sensor setup could not fail, because I built it that way.

A standard trick is to use two sensors, one above the other, and subtract. The background is nearly the same at both, so it cancels, while the nearby defect signal does not.

In my first version, I built the second sensor's reading from the same background arrays as the first. So the background cancelled perfectly. Not because the method is good, but because I had made the two backgrounds identical. The measured difference was pure sensor noise.

Fixing it meant modelling what really differs between two sensors half a metre apart: the Earth's field gradient, and local geology.

Then came the honest result. **The two-sensor version is worse.** Detection contrast is about 3.2 times for a single sensor with background removal, and about 1.45 times for the two-sensor difference. Subtracting cancels the background but adds the noise of both sensors together. After background removal has already done most of the work, you pay the noise cost for a benefit you already had.

That is a negative result about a technique the field treats as standard. It is in the repository, written down as such.

Checking the data before using it

In the lab, my data checking was "look at the plot, and if it looks strange, investigate". That does not work for a pipeline that runs at night on data nobody looks at.

So every survey passes 14 automatic checks at loading time. Seven can stop a survey. Seven only warn. They cover things like schema, physical range, a stuck sensor repeating the same value, GPS jumps, and noise level.

Two decisions here took me time to appreciate.

Do not crash. Set aside. When a survey fails a hard check, the pipeline does not stop. It records all 14 results, marks that survey as quarantined, copies it with a report into a separate folder, and continues with the other surveys.

The reason is practical. A pipeline that stops a whole nightly run because of one bad sensor gets switched off by the people who operate it. Refusing bad data is correct behaviour, and correct behaviour should not look like a crash.

Raw data is never overwritten. Every survey is identified by a hash of its contents. If I load the same file twice, nothing happens. If I load a *different* file under the same name, it stops with an error saying raw data is immutable and it refuses to overwrite.

That error has saved me several times. Data quietly changing under a name I already trained on is a problem you do not notice until your results move and you cannot explain why.

Testing honestly

This is where I changed how I think.

I had far fewer independent samples than rows. My three surveys measure the *same* 12 defects. That is 12 independent objects, not 36 measurements. Every confidence interval in the project resamples whole defects, not rows. If I had resampled rows, I would have produced very narrow and completely false error bars.

I split the data by location, not randomly. Neighbouring measurements are almost identical, so a random split puts nearly the same data in training and testing, and the score looks great for no reason.

I refused to use one popular metric. Only about 3 percent of rows contain a defect. With such unbalanced data, ROC-AUC looks flattering and says little. I use it nowhere.

Then I set quality gates the model must pass before it can be released. The important detail is that almost every gate is measured on the *lower end of the confidence interval*, not the best estimate. A model that wins on average, but not with confidence, has not shown anything.

The model failed, and then I checked whether the failure was real

The result on my first corpus:

```
Recall difference (new model - simple baseline): -0.006 [-0.067, +0.050]
Gate: DID NOT PASS
```

The new model was slightly worse than a simple threshold. But look at the interval. It goes from -0.067 to $+0.050$. It contains zero. With only 60 defects, I could not tell "slightly worse" from "no idea".

A weak negative result is not a result. So the question became: is this real, or is my dataset too small to see it?

I first tried the obvious fix, putting more defects on one line. That made things worse: packing them closer means the background around each defect is polluted by its neighbours. More data, worse data.

The correct fix was more separate lines. I scaled up to 40 lines of 40 km each, three surveys per line. That is **9.6 million rows**, with the same defect density as before.

```
Recall difference (new model - simple baseline): -0.029 [-0.033, -0.026]
Gate: DOES NOT PASS
```

Now the interval is narrow and does not contain zero. With 800 times more defects, the answer is clear. The complex model is genuinely, repeatedly, slightly worse than a simple robust threshold on this data.

The numbers show exactly where it loses.

A real inspection team cannot dig everywhere. They have a fixed budget, so I allow the model 5 holes per kilometre and judge it only on those. Both models get the same number of holes, which makes them directly comparable: whatever they suggest beyond the budget costs nothing and is never checked.

Percent of all holes	Simple threshold	Complex model
Found no defect	23.3	26.6
— landed on buried junk	23.3	23.0
— landed on empty ground	0.0	3.6
Average position error (metres)	0.28	0.42

The two middle rows add up to the first one.

The simple threshold does make mistakes, but every one of them lands on buried junk. That is the honest kind of mistake, and it is exactly the trap I built into the data.

The complex model digs 3.6 percent of its holes in empty ground, where there is nothing at all, and it places every hole about 50 percent less accurately. It does reject junk slightly better, 23.0 against 23.3, but that gain is ten times smaller than what it loses elsewhere.

There is a deeper reason, and it is my mistake in how I framed the problem. I built the interference to have the *same strength* as a defect. So interference is exactly as unusual as a defect. But an anomaly detector looks for things that are unusual. Being unusual is the property the two things **share**. To separate them you need to know that a narrow peak means damage and a wide peak means junk, and that knowledge can only come from labelled examples. I asked an unlabelled method to answer a question that needs labels.

I know the information is there, because my classifier, which does see labels, identifies buried junk with 99.9 percent precision using the same features.

The training command exits with an error code when this gate fails, so the CI job is red. It is supposed to be. A separate CI job runs the code checks and 276 tests, and that one is green. I keep them apart so that a real model failure never looks like broken code, and broken code never hides behind a model excuse.

Nothing has ever been promoted to production status. Every release is still marked "challenger".

Running at scale also found three real bugs

A 0.4 percent data problem caused a 100 percent failure. In 36 of 18,917 cases, a detection landed just outside a defect's labelled zone, so its true severity was missing. One missing value passed into the uncertainty calibration, and because of how the maths propagates, it destroyed the calibration for that entire group. A tiny data issue became a total failure, not a small one.

Some settings did nothing. Two model settings in my configuration file were never actually passed to the model. Changing them had no effect at all, silently, because they happened to match the built-in defaults. I only found this when larger values changed nothing. Fixing it improved crack detection from 0.50 to 0.64.

Settings that work on small data can break on large data. I had reduced the model size so it would work with only 60 defects. Applied unchanged to 9,600 defects, that same limit destroyed the uncertainty calibration completely. Small-data workarounds are not production settings.

What did work

Severity prediction with uncertainty works well. I ask for intervals that contain the true value 90 percent of the time, and measured 92 percent on the small corpus and 89.6 percent on the large one. That is the result I trust most.

Interference rejection works very well. The classifier identifies buried junk with 99.9 percent precision. That was the hard problem I set out to solve.

Crack detection does not reach my target. I required 90 percent, and measured 64 percent. Better than the baseline's zero, but not good enough to release.

Growth prediction passes its gate, but I do not think it means much, and I want to say so. My generator grows defects by exactly 15 percent every time, with no randomness. So a correct estimator recovers that number almost exactly, and mine does. That tells me my arithmetic is right. It does not tell me the method works on real defect growth, which is messy and irregular.

What I learned

The modelling was the smallest part of the work. Feature engineering and model choice were maybe 15 percent. The rest was data contracts, validation, provenance, and the methods for deciding whether a result is real.

Most of that is not new to a physicist. It is laboratory discipline in a different form.

Recording the exact contents of every input file is a lab notebook you cannot backdate. Knowing that 12 defects measured three times is 12 samples and not 36 is just knowing your independent samples. Refusing to release a model that fails its gate is refusing to publish a result that is not significant.

What I had to learn is that in software these habits must be written as code. Gates, checks, and automatic tests. In the lab, I am the check. In a pipeline that runs at night on data nobody reads, the check must be code, or it does not exist.

The most useful thing I built is a job that turns red when the model is not good enough. Not because red is good, but because a system that can only say yes tells you nothing when it says yes.

My detection model does not beat a simple threshold. I know this with a confidence interval of -0.033 to -0.026 , measured over 9,600 defects. I know why. And it is written down in the model card, the README, and a red CI badge, instead of in a drawer.

The full project is on GitHub at [Alibaghizadeh/lsm-integrity](https://github.com/Alibaghizadeh/lsm-integrity). The data is synthetic. The results are real, including the ones that did not pass.